



OUTLIERS

ProjectForge — A Work Board For All Your Agents

Cards with an owner, written work reports, and a single orchestrator that moves the cards

PIECE 3 OF 4 • THE AGENT WORKSPACE

DOWNLOAD

<https://github.com/OUTLIERS-ai/outliers-ws-q3-projectforge>

EST. MMXXVI • OUTLIERS ACCELERATOR

What it is

ProjectForge is a work board, like a Trello board, that runs on your own computer and is written for AI agents. Each card is a single piece of work. Each card has an owner: one of your Claude Code agents, or you.

The board has up to 8 columns. **Awaiting You** comes first, because it is the only column that needs you. Then Backlog, Ready, In Progress, Blocked, Review and Done. An 8th column, Tracking, appears once another program sends it facts to show. On the right is a live list of who did what and when, and refused changes show there in red.

ProjectForge
// Sam Carter Bookkeeping - demo

3 PROJECTS 10 OPEN 1 AWAITING YOU 3 REFUSED TODAY 1 ALERTS

+ Project All Content Sales Client work

Search cards... Command Deck Board Queue Agents ?

AWAITING YOU 1

- Call back Dan at Oakfield Joinery about payroll
Follow-ups
you just now

BACKLOG 2

- Pick the lead magnet: checklist or calculator
Year-end tax season campaign
unassigned just now
- Schedule the 3 posts
Year-end tax season campaign
social-poster just now

READY 4

- Email to past enquiries: book before the rush
2026-10-02 1/3
Year-end tax season campaign
writer-bot just now
- Set up Priya Shah Design's bank feeds
Client onboarding
inbox-helper just now
- Sign off the quarterly VAT figures
Client onboarding
you just now
- Tidy the client folder names
Client onboarding
unassigned just now

IN PROGRESS 1

- Draft the welcome pack for new clients
WRITER-BOT WORKING...
Client onboarding
writer-bot O1 just now

BLOCKED 1

- Oakfield Joinery: chase missing VAT receipts
Client onboarding
inbox-helper O1 just now

REVIEW 1

- Write: trader:
Year-end tax season campaign
editor

ALERTS (1)

ACTIVITY show refusals only

- you edited priority
2026-09-22 14:09:50
- you set tags: none
2026-09-22 14:09:50
- NEEDS A PERSON writer-bot: I need the client's logo file to finish the welcome pack
2026-09-22 14:09:43
- REFUSED writer-bot: handover refused: too thin done: 'handover' does not say what was done (name the files); missing decisions: decisions made and why; missing state: where the work stands, including what is NOT done; missing next_first: what the next agent should do first; missing warnings: warnings (write 'none' if there are none)
2026-09-22 14:09:43
- REFUSED writer-bot: refused: writer-bot is a worker and may not move a card between columns. Workers only append: log a pass, write a handover or a comment. The orchestrator moves cards; a manager opens them.
2026-09-22 14:09:43
- REFUSED writer-bot: refused: writer-bot is a worker and may not open a card. Workers only append: log a pass, write a handover or a comment. The orchestrator moves cards; a manager opens them.
2026-09-22 14:09:43
- you set tags: check-me
2026-09-22 14:01:34
- you edited priority
2026-09-22 14:01:34

1 Awaiting You comes first. Only you take a card out of it.
2 + Project: make a project first. Every card lives in one.
3 Changes the board refused today. Click to list them.
4 Who may do what: the rules, on screen.
5 A refused change, in red, with the agent's name.

THE BOARD WITH MADE-UP WORK FOR A MADE-UP BOOKKEEPING BUSINESS. THE NUMBERED YELLOW MARKS ARE EXPLAINED IN THE BOX.

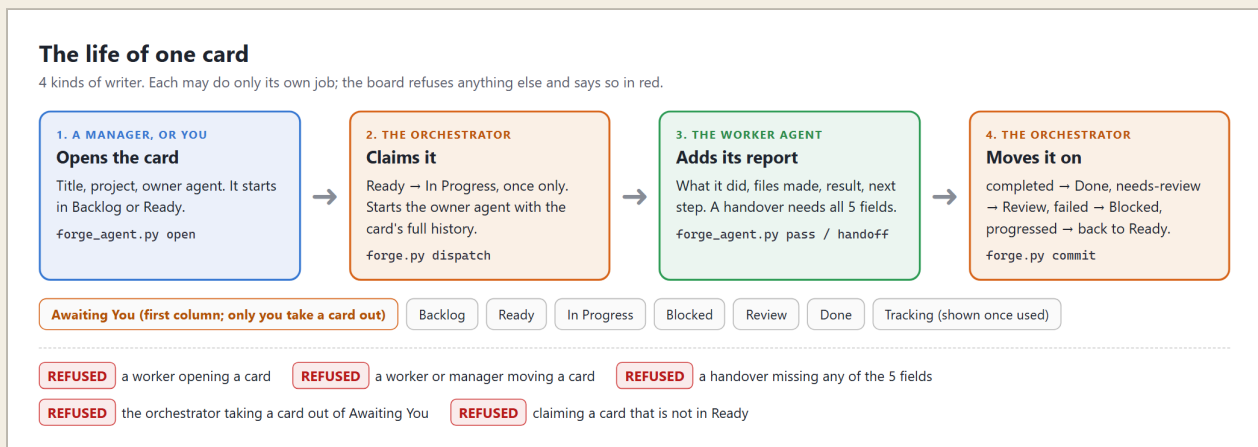
Click a card and you see its full record: notes, checklist, a link to a person in your CRM vault, every work report an agent wrote against it (what it did, which files it made, the result, the next step), and every handover from agent to agent.

The board itself contains no AI. It stores the work and it enforces these rules:

- Managers open cards.** You choose which of your agents are managers. Every other agent is a worker.
- Workers only add to a card.** A worker can write a work report, a handover, a comment or an escalation (a flag saying it needs a person). It can never open a card or move one.
- Only the orchestrator moves cards.** The orchestrator is a single Claude Code session, started by the `/forge-run` command. It is the only agent allowed to move a card between columns. You can move cards too.

4. **Awaiting You is yours.** The orchestrator may put a card into it. Only you take a card out.

A handover between agents is refused unless it carries 5 fields: what was done, the decisions and why, where the work stands, what to do first, and warnings.



THE LIFE OF A CARD: WHO MAY DO WHAT AT EACH STEP, AND WHAT THE BOARD REFUSES.

PIECE 3 • PROJECTFORGE

Why you would want it

Once you have more than a handful of agents, 3 problems appear.

- **Nobody can see what happened.** An agent finished a job on Tuesday. On Friday you, or another agent, have no written record of which files it made or what it left undone. The board keeps that record on the card.
- **Agents trip over each other.** An agent marks another agent's work as finished, or 2 runs pick up the same job. Only the orchestrator moves cards, so "this is now done" is decided in a single place, and a card can be claimed only while it sits in Ready, so a second `/forge-run` cannot claim it again.
- **Handovers are empty.** "Handed over to the editor" tells the editor nothing. The 5-field handover makes the first agent write down what the second agent needs.

It also gives you a column that is yours, Awaiting You, and anything that would reach another person (a post, an email, a message) stops in Review for you to check. The rules are on screen: click ? at the top right.

Who may do what

×

The board checks every change against these rules and refuses the rest. Every refusal is shown in red in the Activity list, with the agent's name.

WHO	MAY	MAY NOT
YOU you	anything: open, edit, drag, close cards	-
MANAGER content-lead	open new cards, edit card details, add reports	move a card between columns
WORKER every other agent	add a work report, a 5-field handover, a comment, an escalation	open a card, move a card, edit a card
ORCHESTRATOR the /forge-run command	claim a Ready card, record the result and move the card; put a card into Awaiting You	take a card out of Awaiting You

Nothing moves on its own. Type `/forge-run` in Claude Code to hand Ready cards to their agents (`/forge-run dry` previews and changes nothing).

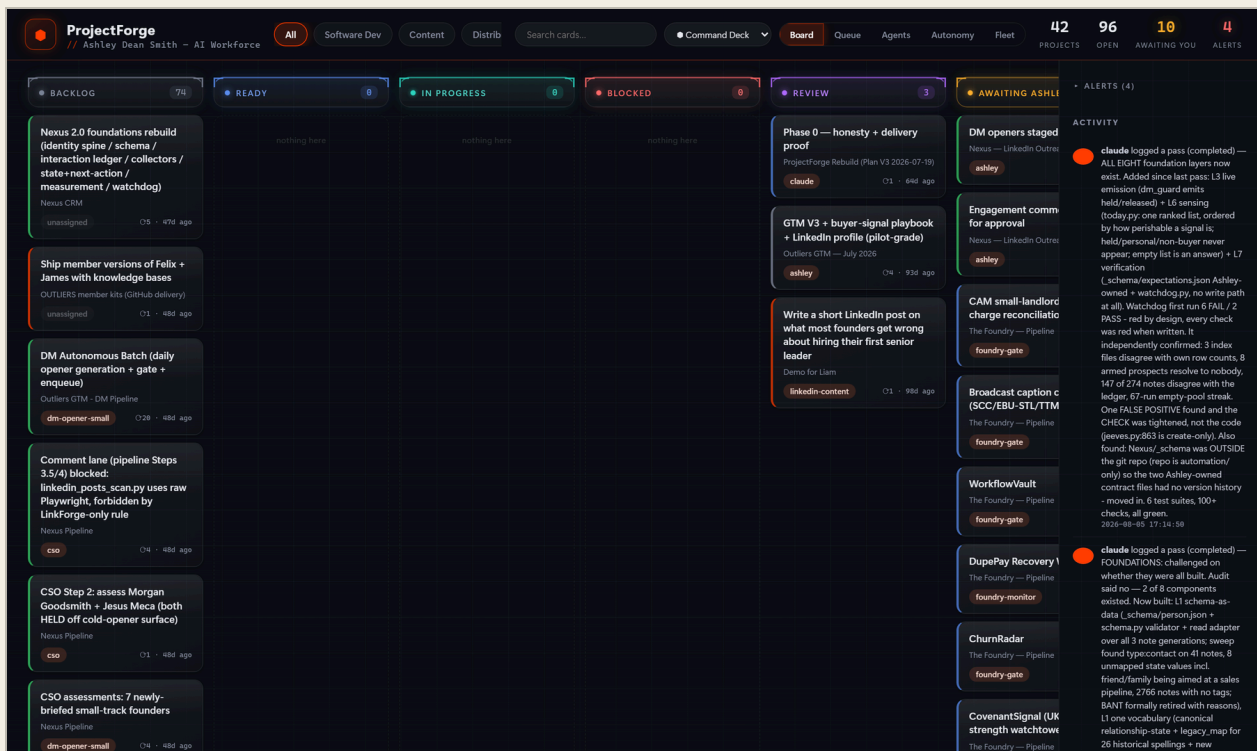
An example refusal: `refused: writer-bot is a worker and may not move a card between columns.`

THE "WHO MAY DO WHAT" WINDOW, OPENED WITH THE ? BUTTON. IT LISTS YOUR MANAGERS BY NAME.

PIECE 3 • PROJECTFORGE

How we built it

Ashley built the original for his own agents. Every date and number below comes from his build records. The pictures in this section are his real board, captured on 2026-09-22 from a copy of his database (the data was last written on 2026-08-07).



ASHLEY'S REAL BOARD: 42 PROJECTS, 96 OPEN CARDS, 10 AWAITING HIM, 4 ALERTS. 74 CARDS SIT IN BACKLOG AND 0 IN READY OR IN PROGRESS: NOTHING WAS BEING HANDED OUT.

2026-06-12: THE BOARD IN 1 DAY

Ashley built ProjectForge in a single day as a "work operating system" for his agents. It had a SQLite database (a single file on disk), a web server on port 3020 written in plain Python with no extra libraries, a hand-written JavaScript board, and a markdown copy of the board saved into his vault. He kept the engine code apart from his own settings so it could be packaged for other people. This download is that packaging.

The same day he added an always-on background program that checked the board every 15 minutes for overdue, stale, stuck and unowned cards, and archived cards done more than 14 days earlier. It used no AI.

2026-06-13: OTHER SYSTEMS FEED IN, AND THE ORCHESTRATOR LOOP

Small scripts, called adapters, read Ashley's other systems and pushed their work onto the board every 30 minutes. Re-running never created copies: each card was matched on the source program plus that program's own ID.

Then the orchestrator loop: `/api/next` ranks the ready cards, `/api/dispatch` claims a card (Ready to In Progress), and `/api/commit` records the agent's report and moves the card. The `/forge-run` command in Claude Code runs that loop.

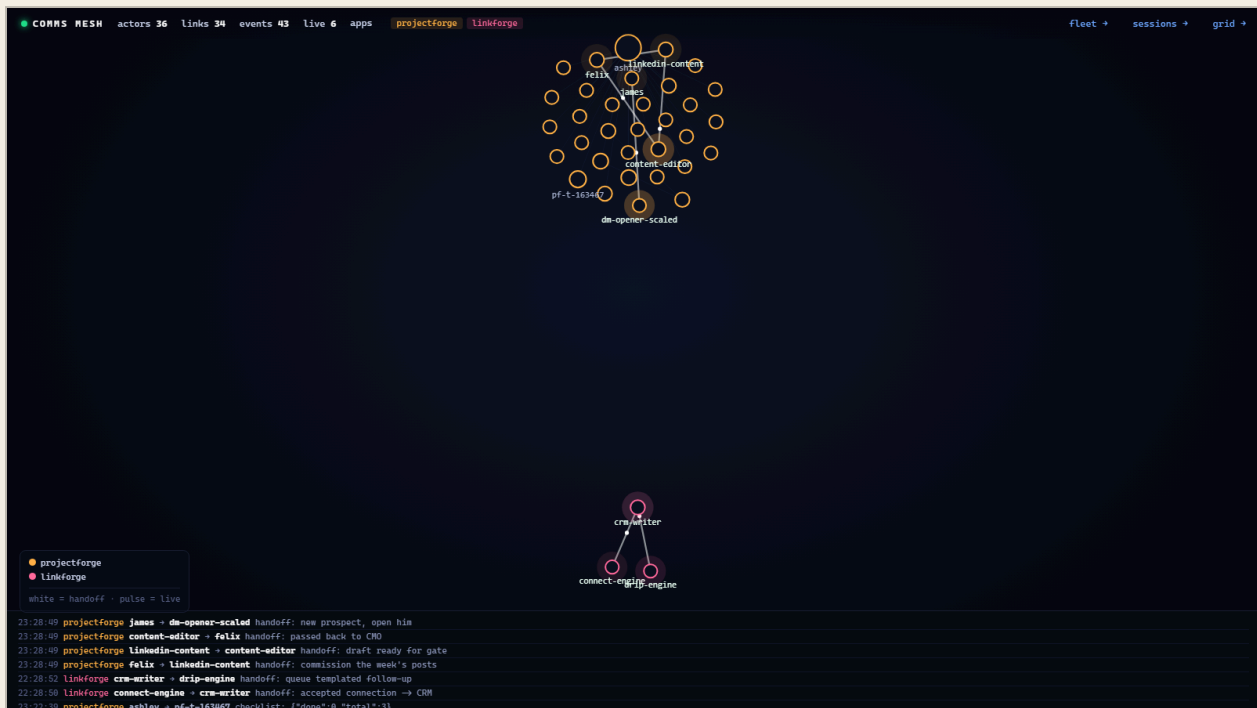
He also switched on an autonomous mode: a Windows Scheduled Task that started `claude -p "/forge-run"` with no window every 15 minutes.

2 bugs appeared that day and were fixed:

- The limit of 10 cards in progress was filled by 18 cards that only mirrored facts from other systems, so no real card could ever be handed out. The fix was an 8th column, **Tracking**, that never counts

against the limit.

- A card the orchestrator had just claimed could be dragged back to its old column by the next 30-minute sync. The fix was a flag on the card: once the orchestrator has claimed it, a sync may no longer change its column.

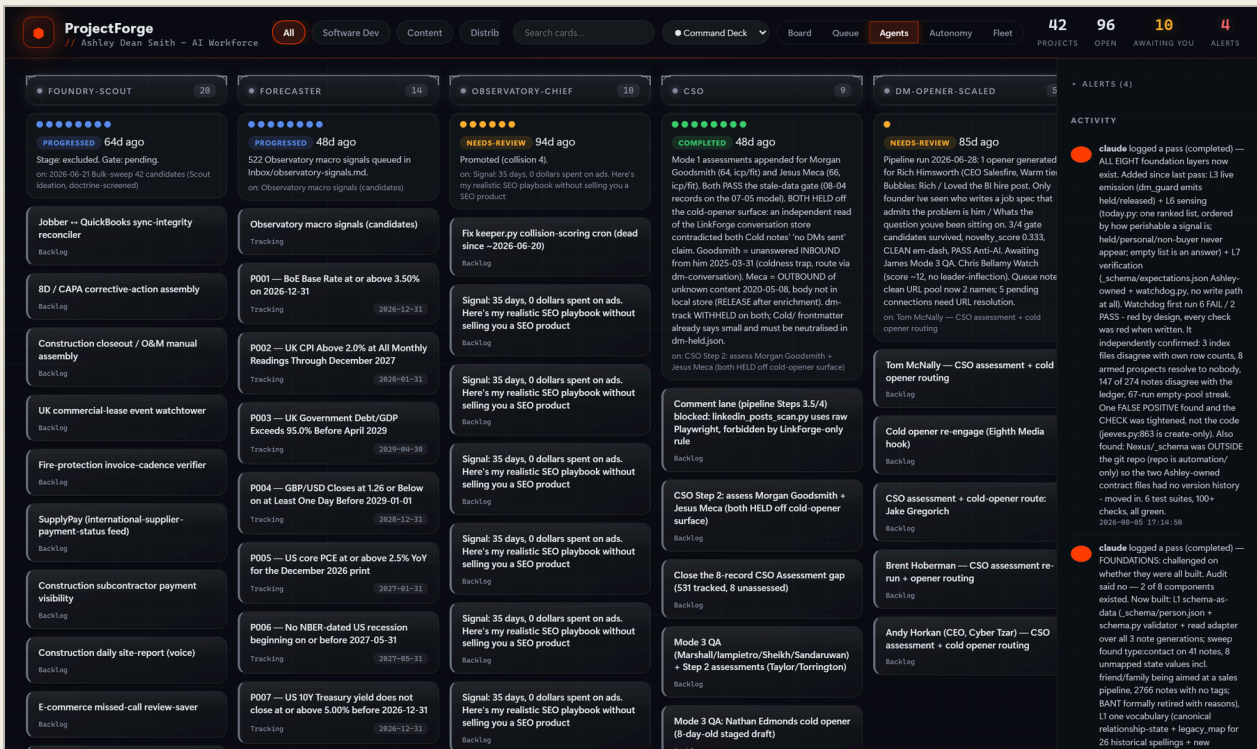


ASHLEY'S COMMS MESH ON 2026-06-12: PROJECTFORGE HANDOVERS BETWEEN HIS AGENTS DRAWN AS LINES, WITH THE HANDOVER LIST ALONG THE BOTTOM.

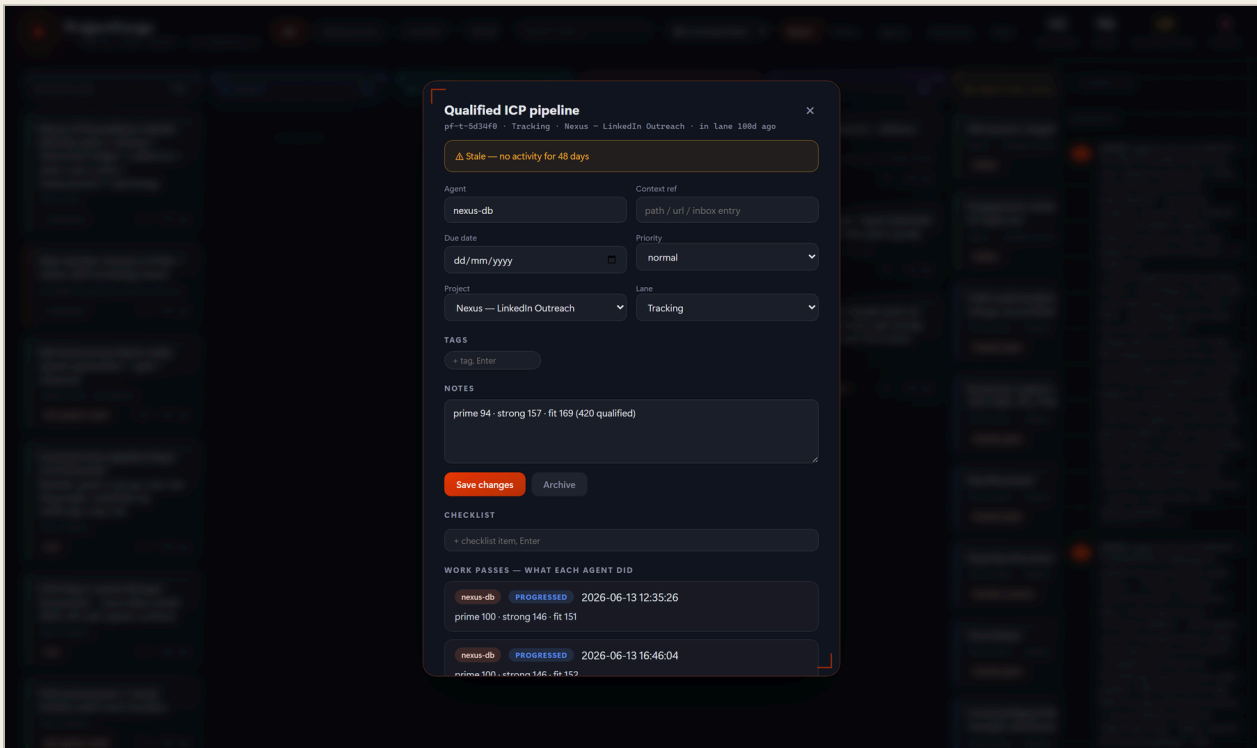
2026-06-14: GIT, A DESKTOP APP, AND THE SINGLE-WRITER RULING

The code went into git (33 files, 5,011 lines) and was packaged as a desktop app. Ashley then set the rule this download is built around: managers open cards, workers only add reports, handovers and comments, and only the orchestrator moves a card. The shared tool `forge_agent.py` was built so every agent writes to the board the same way.

He also measured a 7-day starting point: **3 cards done, 18 times Ashley stepped in, 4 cards handed out.** That told him the automatic loop was not yet earning its keep.



ASHLEY'S AGENTS TAB: OPEN CARDS PER AGENT (FOUNDRY-SCOUT 20, FORECASTER 14, OBSERVATORY-CHIEF 10, CSO 9), EACH WITH ITS LAST WORK REPORT AND A ROW OF RESULT DOTS.



A REAL CARD: "QUALIFIED ICP PIPELINE", THE CARD WITH THE MOST WORK REPORTS ON HIS BOARD (65). THE YELLOW BAR SAYS NO ACTIVITY FOR 48 DAYS.

2026-07-08: THE TIMER WAS SWITCHED OFF

The board cost nothing to run. The timer did. Every 15 minutes Windows started a fresh Claude session to run `/forge-run`, whether or not there was work, and each start paid for a full session before looking at the board. The board was usually empty.

The surviving log shows **304 passes between 2026-07-04 and 2026-07-08, about 69 a day, and every one found 0 cards to hand out.** Of 40 passes sampled, 0 handed out any work. Ashley's records put the total at about 230 million input tokens over 30 days (that figure is from his notes, not re-measured). On 2026-07-08 he switched it off completely.

His one-line summary: a free board wearing an expensive alarm clock. The fix: never start Claude unless a cheap check shows a card is ready.

ASHLEY'S AUTONOMY TAB TODAY: DISARMED, THE WINDOWS TASK DISABLED, LAST RUN 08/07/2026 11:55 (A UK DATE: 8 JULY), AND THE LAST PASS REPORTING NOTHING TO HAND OUT.

NOTE

On 2026-07-19 a single orchestrator pass was measured on Anthropic's smallest model at that time, run from a folder with no list of agents loaded: it cost \$0.094. The cost of a pass depends heavily on how many agents and instructions each session loads.

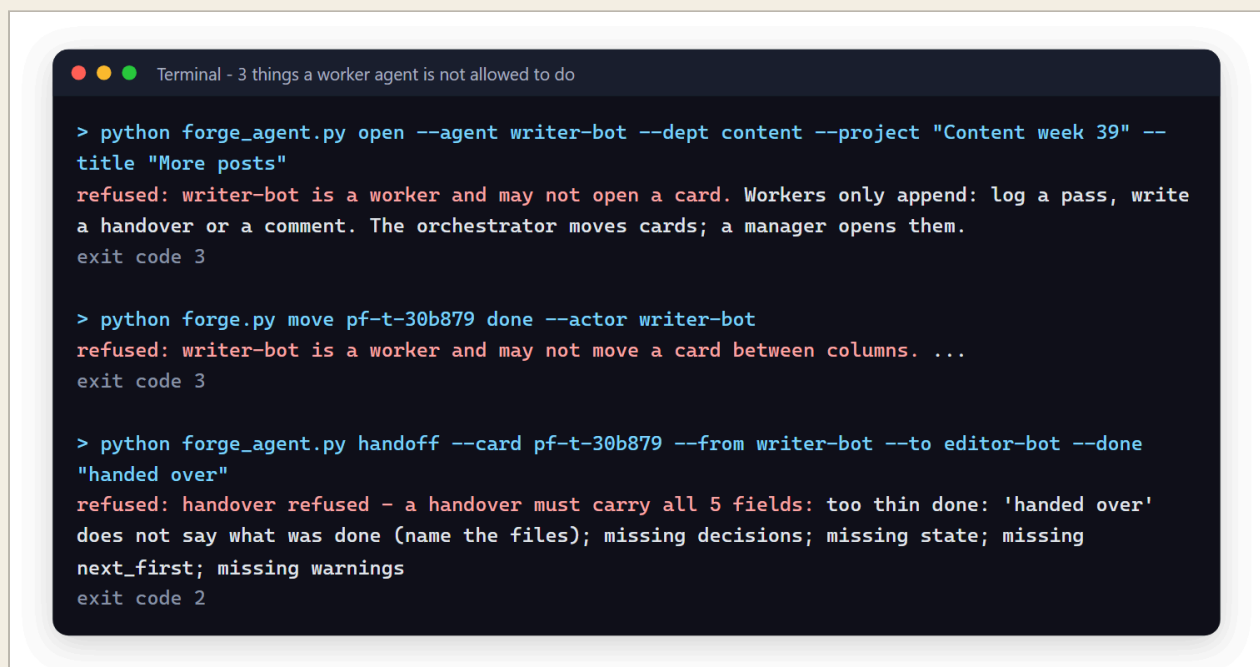
2026-07-19: THE REBUILD PLAN AND THE HANDOVER STANDARD

Ashley wrote a new plan in a single day, in 3 rounds: version 1, a critique, version 2, a critique, version 3. The aim: agents that work like independent employees, with Ashley needed "barely ever". The same evening he ruled that "handoff occurred at [time]" is not a handover, and set the 5 fields. The first stage of the plan made timestamps honest (a sync no longer counts as activity) and made every write name who made it.

The plan said the database should refuse a thin handover. **That refusal was never built** in the original. The rule only worked when the person recording the work remembered it.

2026-09-22: WHAT CHANGED FOR THIS DOWNLOAD

- **The handover refusal is built.** The board refuses a handover missing any of the 5 fields, and the decisions field must say why (it must contain "because", or say "none").
- **The rules are checked inside the database code**, not only in the agents' instructions, and every refusal is written to the Activity list in red, with the agent's name.
- **No timer by default.** You run `/forge-run` when you want it. The optional schedule runs a Python check first and exits without starting Claude when nothing is ready.
- **Agents write straight into the database**, so their reports are kept even when the web board is closed. In the original, a report sent while the board was down was not recorded.
- **Safety fixes found in testing on 2026-09-22**, before any member had it: a website you visit can no longer write to the board; every `forge.py` change must name who made it; the orchestrator can no longer take a card out of Awaiting You; a card cannot be claimed twice; a second board on the same port now fails with a message instead of starting silently.
- The "Awaiting Ashley" column is now "Awaiting You". Every folder path comes from a single settings file the installer writes. The desktop app, the adapters written for Ashley's own systems and the self-repair step were left out.

A terminal window titled "Terminal - 3 things a worker agent is not allowed to do" with a dark background and light blue text. It shows three failed commands and their corresponding refusal messages in red text.

```
> python forge_agent.py open --agent writer-bot --dept content --project "Content week 39" --title "More posts"
refused: writer-bot is a worker and may not open a card. Workers only append: log a pass, write a handover or a comment. The orchestrator moves cards; a manager opens them.
exit code 3

> python forge.py move pf-t-30b879 done --actor writer-bot
refused: writer-bot is a worker and may not move a card between columns. ...
exit code 3

> python forge_agent.py handoff --card pf-t-30b879 --from writer-bot --to editor-bot --done "handed over"
refused: handover refused - a handover must carry all 5 fields: too thin done: 'handed over' does not say what was done (name the files); missing decisions; missing state; missing next_first; missing warnings
exit code 2
```

A WORKER TRYING TO OPEN A CARD, A WORKER TRYING TO MOVE A CARD, AND A THIN HANDOVER. THE BOARD REFUSES ALL 3, WITH EXIT CODES YOUR SCRIPTS CAN CHECK.

Pros and cons

	FOR	AGAINST
Cost	The board, the health check and the CRM reader use no AI and cost nothing to run.	Each <code>/forge-run</code> is a full Claude Code session. Ashley's timer ran about 69 of them a day for no work. Only run it when something is ready.
Visibility	A single place to see what every agent did, on which job, with which files and what comes next.	Agents only write to it if their instructions say so. Before 2026-06-14, 0 of Ashley's 77 agent files wrote to his board.
Safety	Only the orchestrator moves cards; only you take a card out of Awaiting You; outward work stops in Review; refusals show in red.	It is a guard rail, not a lock. An agent that gives a false name (for example <code>--actor you</code>) gets through. The rules stop honest mistakes, not a determined liar.
Handovers	Thin handovers are refused at the database.	Agents need a few extra lines of instruction to write good ones.
Busy-work	A queue makes waiting work obvious.	A board that hands out work invites agents to create work. Ashley's first week: 3 cards done against 18 times he stepped in. His board on 2026-09-22 had 74 cards in Backlog and 0 being worked.
Size	About 5,000 lines of plain Python and plain JavaScript. Your own Claude can read and change all of it.	With 3 to 5 agents, a simple list in your vault may be enough. The board pays off when agents hand work to each other.
Time	Install takes about 5 minutes.	Wiring your agents in (the lines for CLAUDE.md and each agent file) takes 20 to 30 minutes.

Why nothing runs on a timer

What Ashley's timer did in July 2026, and what the download does instead.

THE ORIGINAL: EVERY 15 MINUTES

Start Claude, then look

A Windows task started a full Claude Code session to run `/forge-run`, whether or not there was work.

304 starts

between 2026-07-04 and 2026-07-08, about 69 a day.

0 of them found a card to hand out. Switched off on 2026-07-08.

THIS DOWNLOAD: OFF UNLESS YOU SWITCH IT ON

Look first, with no AI

```
python tools/run_if_ready.py
```

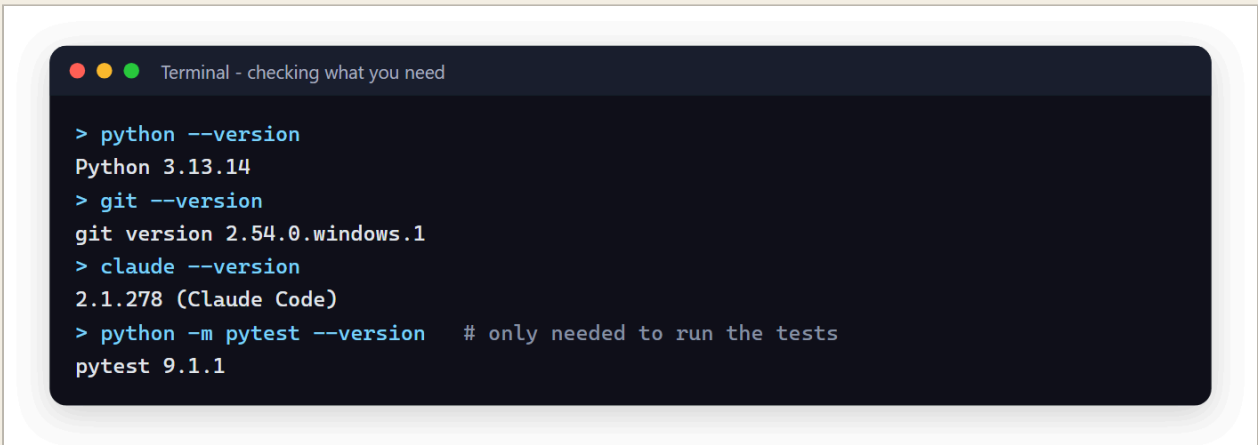
- Counts the cards that could be handed out, straight from the database (a few milliseconds, no Claude).
- 0:** writes 1 line to `data/run_if_ready.log` and stops. Claude is never started.
- 1 or more:** starts `claude -p "/forge-run"` in "dontAsk" mode, allowed only the board commands it needs.

WHAT A TIMER COST ASHLEY, AND WHAT THE DOWNLOAD DOES INSTEAD.

Before you start

YOU NEED	HOW TO CHECK
Python 3.9 or newer (tested on 3.13)	Open a terminal and type <code>python --version</code> . On a Mac use <code>python3 --version</code> .
Git	<code>git --version</code>
Claude Code, logged in	<code>claude --version</code> . Tested on version 2.1.278 on 2026-09-22.
Your second brain vault	You know its folder, for example <code>C:\Users\<you>\Documents\Second Brain</code> .
Your agents folder	Usually <code>C:\Users\<you>\.claude\agents</code> (Windows) or <code>~/.claude/agents</code> (Mac).
Optional: your CRM vault	The folder that contains <code>Today.md</code> and <code>People/</code> .
Optional, for the tests: pytest	<code>python -m pytest --version</code> . If missing: <code>python -m pip install pytest</code> .

Node.js is not needed. Nothing is installed from the internet: the board uses only the Python standard library.



THE 4 CHECKS, AS THEY LOOKED ON THE TEST COMPUTER ON 2026-09-22.

Install it

- 1. Open a terminal in the folder where you keep your tools, for example your Documents folder.
- 2. Download the code and start the installer:

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-03-projectforge && cd outliers-ws-03-projectforge && python install.py
```

3. Answer the 8 questions. Most offer a default in square brackets; press Enter to accept it. Questions 4 and 5 have no default: press Enter for nobody.
 - Your second brain vault folder.
 - Your CRM vault folder, or `none`.
 - Your agents folder. The installer lists every agent it finds.
 - Which agents are **managers** (may open cards). Press Enter and only you open cards.
 - Which agents send work to other people. Their finished cards stop in Review.
 - What the board calls you (default `you`). Remember it: you type it after `--actor` in terminal commands.
 - Where the `/forge-run` command goes: `user` (every Claude Code session) or `vault` (only sessions in your second brain).
 - The board summary note: pressing Enter writes it into your vault at the path shown. Type `none` if you do not want one.
4. The installer shows every file it is about to write and asks "Go ahead?". Type `y`.
5. When it worked you see a list of `done:` lines like this:

```

Terminal - python install.py (yellow = what Sam typed)

> python install.py
ProjectForge installer - a work board for your agents.

1. Your second brain vault (the Obsidian folder your agents work in).
  Path [C:\Users\sam\Documents\Second Brain]: <Enter>
2. Your CRM vault (type 'none' if you do not have one).
  Path [C:\Users\sam\Documents\CRM]: <Enter>
3. Your agents folder(s). Separate several with ;
  Folders [C:\Users\sam\.claude\agents]: <Enter>
  Found 4 agent(s) who can own cards:
    content-lead, editor-bot, research-bot, writer-bot

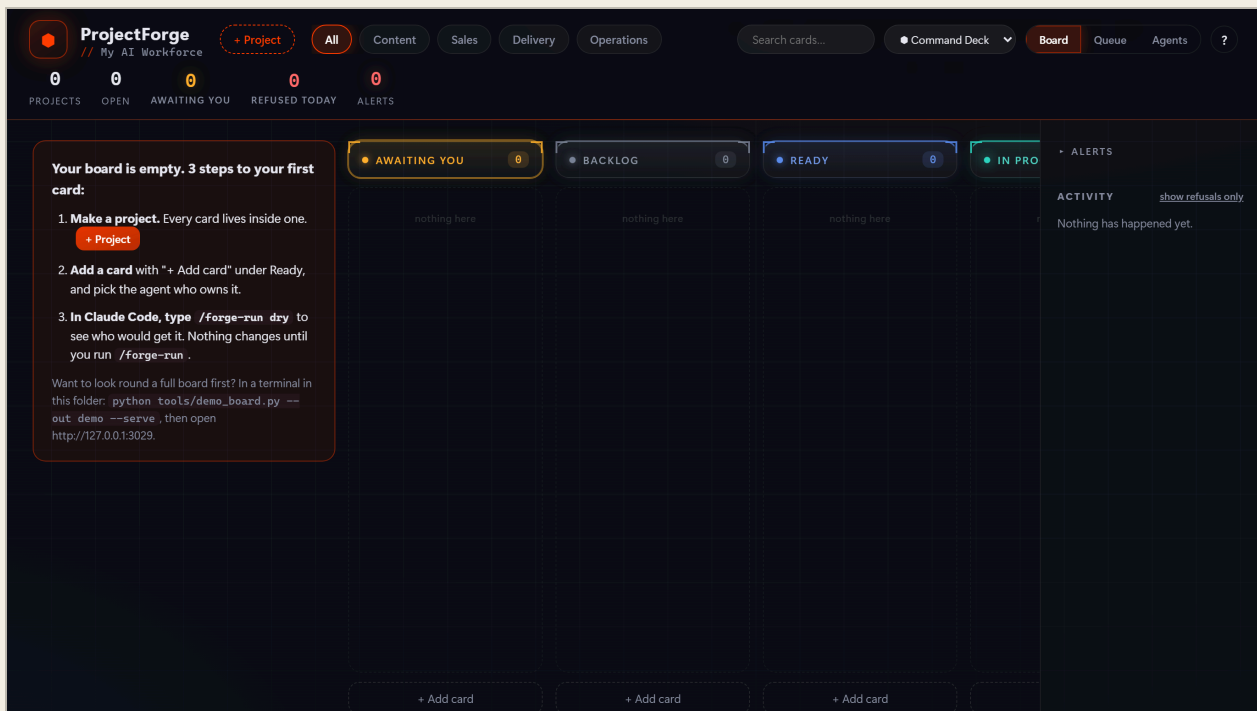
4. Managers: which agents may OPEN new cards? Everyone else may only add work reports.
  Names separated by commas. No default: press Enter for none (then only you open cards).
  Managers: content-lead
5. Which agents' work reaches other people (posts, emails, messages)?
  Their finished cards stop in Review for you to check. Commas. No default: press Enter for none.
  Outward agents: editor-bot
6. What should the board call you?
  Your name on the board [you]: Sam
7. Where should the /forge-run command go?
  user = every Claude Code session on this computer;
  vault = only sessions opened in your second brain.
  user or vault [user]: <Enter>
8. A board summary note in your second brain (a markdown copy of the board,
  rewritten after every change). Press Enter to write it at the path shown,
  or type 'none' for no note.
  Note path [C:\Users\sam\Documents\Second Brain\ProjectForge Board.md]: <Enter>
About to:
write C:\Users\sam\Documents\outliers-ws-03-projectforge\config.json
create an empty board in C:\Users\sam\Documents\outliers-ws-03-projectforge\data (if not already there)
install C:\Users\sam\.claude\projectforge\forge_agent.py
install C:\Users\sam\.claude\commands\forge-run.md (backup first if different)
write C:\Users\sam\Documents\Second Brain\ProjectForge Board.md (only if it does not exist)
Go ahead? [Y/n]: y
done: config.json created
done: empty board created at C:\Users\sam\Documents\outliers-ws-03-projectforge\data\forge.db
done: forge_agent.py created
done: forge_client.py created
done: forge_agent.json created
done: /forge-run command created
done: C:\Users\sam\Documents\outliers-ws-03-projectforge\data\CLAUDE-snippet.md created
done: summary note written: C:\Users\sam\Documents\Second Brain\ProjectForge Board.md

Next:
1. Open the board:      python forge.py serve
   then visit         http://127.0.0.1:3020
   (leave that window open; Ctrl+C in it stops the board)
2. Paste the lines in C:\Users\sam\Documents\outliers-ws-03-projectforge\data\CLAUDE-snippet.md
   into the CLAUDE.md every session reads (C:\Users\sam\.claude\CLAUDE.md),
   or your vault's CLAUDE.md if you chose "vault" at question 7.
3. In Claude Code, try /forge-run dry (a preview that changes nothing).
Nothing runs on a timer. To add the optional schedule later:
python tools/schedule.py --print

```

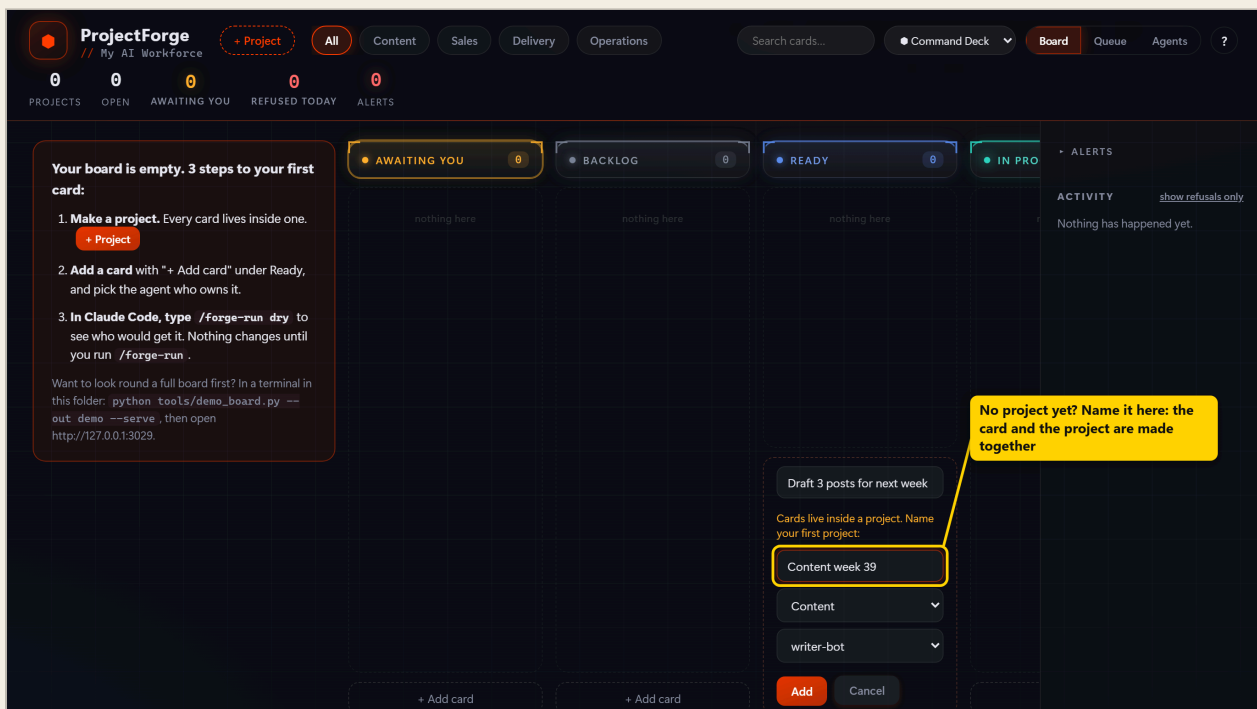
A FINISHED INSTALL FOR A MADE-UP MEMBER CALLED SAM. YELLOW IS WHAT SAM TYPED.

- Open the board: `python forge.py serve`, then visit `http://127.0.0.1:3020` in your browser. There is no login and no wait. The terminal prints the address and then stays quiet while the board runs. That is normal. Leave the window open; press Ctrl+C in it to stop the board. The name under "ProjectForge" comes from `workspace` in `config.json`; change it there.



A FRESH BOARD: EMPTY, WITH THE 3 FIRST STEPS ON THE LEFT AND AWAITING YOU AS THE FIRST COLUMN.

- The board opens empty, with 3 steps on the left. Your first card: click "+ Add card" under Ready. Because there is no project yet, the form asks you to name one, and the card and the project are made together.



A FRESH BOARD ON A 1366-WIDE LAPTOP SCREEN, WITH THE FIRST CARD BEING ADDED.

- Open the file `data/CLAUDE-snippet.md` inside the download folder. Paste the first part into the `CLAUDE.md` that every session reads, `C:\Users\<you>\.claude\CLAUDE.md` (or the `CLAUDE.md` in your vault if you chose `vault` at question 7). Paste the second part into each worker agent's file. This is what makes your agents log their work.

9. In Claude Code, type `/forge-run dry` . It reports what it would do and changes nothing.

TIP

Want to see a full board first? Run `python tools/demo_board.py --out demo --serve` and open `http://127.0.0.1:3029` . It builds a separate demo board of made-up work, with a made-up CRM vault, and does not touch your real one.

NOTE

If a file called `forge-run.md` already exists in your commands folder, the installer copies it to `forge-run.md.bak-<date>` before replacing it. Running the installer a second time with the same answers changes nothing. `python install.py --uninstall` asks you to type y, then takes the command out, moves the agents' tool into a folder named `projectforge.removed-<date>` , puts your own old command back (even if you installed twice), and leaves your board data alone.

PIECE 3 • PROJECTFORGE

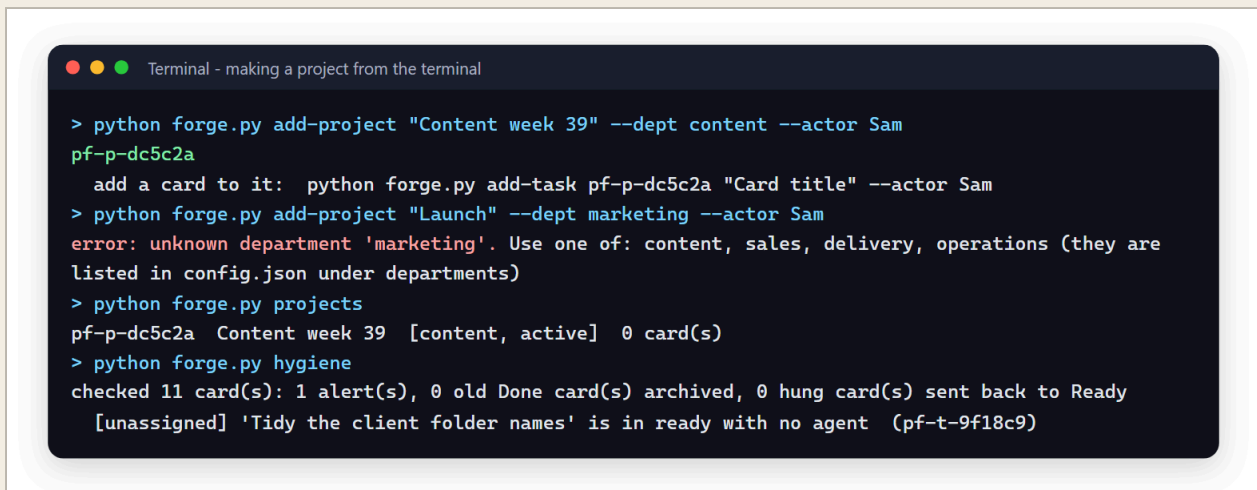
Using it day to day

Adding work on the board. "+ Project" sits at the top of the screen. "+ Add card" sits at the foot of each column, with a project picker and an owner picker. The owner picker only lists you and your real agents, so a typo cannot create a made-up agent.

Adding work from the terminal. Make a project first; it prints the project's id. Then add a card to it. Every command that changes the board ends with `--actor` and your board name (the answer to question 6; `you` if you kept the default):

```
python forge.py add-project "Content week 39" --dept content --actor you
python forge.py add-task pf-p-5baff3 "Draft 3 posts" --status ready --agent writer-bot --actor you
```

The departments are content, sales, delivery and operations; any other is refused. `python forge.py projects` lists every project with its id.



```
Terminal - making a project from the terminal

> python forge.py add-project "Content week 39" --dept content --actor Sam
pf-p-dc5c2a
  add a card to it:  python forge.py add-task pf-p-dc5c2a "Card title" --actor Sam
> python forge.py add-project "Launch" --dept marketing --actor Sam
error: unknown department 'marketing'. Use one of: content, sales, delivery, operations (they are
listed in config.json under departments)
> python forge.py projects
pf-p-dc5c2a  Content week 39  [content, active]  0 card(s)
> python forge.py hygiene
checked 11 card(s): 1 alert(s), 0 old Done card(s) archived, 0 hung card(s) sent back to Ready
[unassigned] 'Tidy the client folder names' is in ready with no agent  (pf-t-9f18c9)
```

MAKING A PROJECT FROM THE TERMINAL, A DEPARTMENT THE BOARD REFUSES, THE PROJECT LIST, AND THE HEALTH CHECK LISTING A CARD.

What your agents run. The installer puts the agents' tool, `forge_agent.py`, in the `.claude\projectforge` folder inside your home folder. It is not in the download folder, so to try it yourself give its full path:

```
python "%USERPROFILE%\claude\projectforge\forge_agent.py" open --agent content-lead --
dept content --project "Content week 39" --title "Write the newsletter" --assignee writer-
bot
```

On a Mac the path is `~/claude/projectforge/forge_agent.py`. The full list of its commands is under "Every command and setting" below.

Letting the agents work. When cards are sitting in Ready, type `/forge-run` in Claude Code (or `/forge-run 3` to run at most 3 cards). The orchestrator:

1. checks how many cards can be handed out, and stops straight away if the answer is 0;
2. moves Backlog cards that have an owner to Ready. It gives an ownerless card an owner only if you set up keyword routing (idea 6 below); otherwise the card waits for you to pick an owner;
3. for each ready card: claims it, starts the owner agent with the card's full history, records the agent's report, and moves the card to the column its result points to. "progressed" puts the card back in Ready for the next run.

WARNING

Backlog is not a waiting room. Any Backlog card with an owner is handed out on the next `/forge-run`. To park a card, give it the tag `someday` (or `icebox` or `parked`).

Reading a card. Click anywhere on it. The top shows the owner, the CRM person link and the checklist. Further down are the work reports and the handovers, each with its 5 fields. A handover also makes the receiving agent the card's new owner. Edits at the top are kept when you add a tag, tick the checklist or comment; if you press Escape with unsaved edits, the board asks first. The board refreshes every 10 seconds, but never while you are typing.

WORK REPORTS — WHAT EACH AGENT DID

writer-bot

NEEDS-REVIEW

2026-09-22 14:00:59

Drafted 3 posts on the receipts sole traders forget: mileage, home office, phone.

Content/Drafts/receipts-post-1.md

Content/Drafts/receipts-post-2.md

Content/Drafts/receipts-post-3.md

→ next: Editor checks tone and the figures on post 2

HANDOVERS — 5 FIELDS EACH

writer-bot

→

editor-bot

2026-09-22 14:00:59

DONE

DECISIONS

WHERE IT STANDS

DO FIRST

WARNINGS

Wrote Content/Drafts/receipts-post-1.md, -2.md, -3.md

Left out the tax-rate numbers because they change in April and would date the posts

3 drafts written. No images yet. Nothing scheduled.

Check post 2: the mileage rate needs a source

Post 3 names a real software brand - check you are happy to mention it

AGENT TRACK

content-lead

originated

2026-09-22 14:00:59

↓

editor-bot

DONE: Wrote Content/Drafts/receipts-post-1.md, -2.md, -3.md | NEXT: Check post 2: the mileage rate needs a source

2026-09-22 14:00:59

COMMENTS

No comments yet.

Write a comment...

Comment

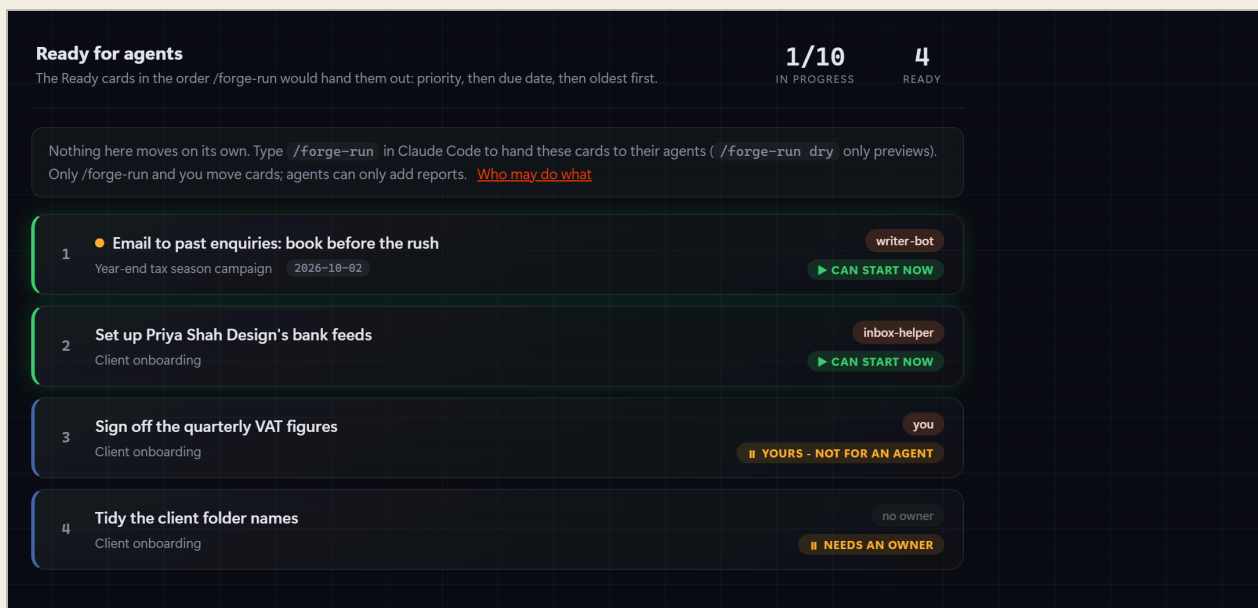
TIMELINE

2026-09-22 14:00:59 content-lead created this

2026-09-22 14:00:59 orchestrator handed this to writer-bot

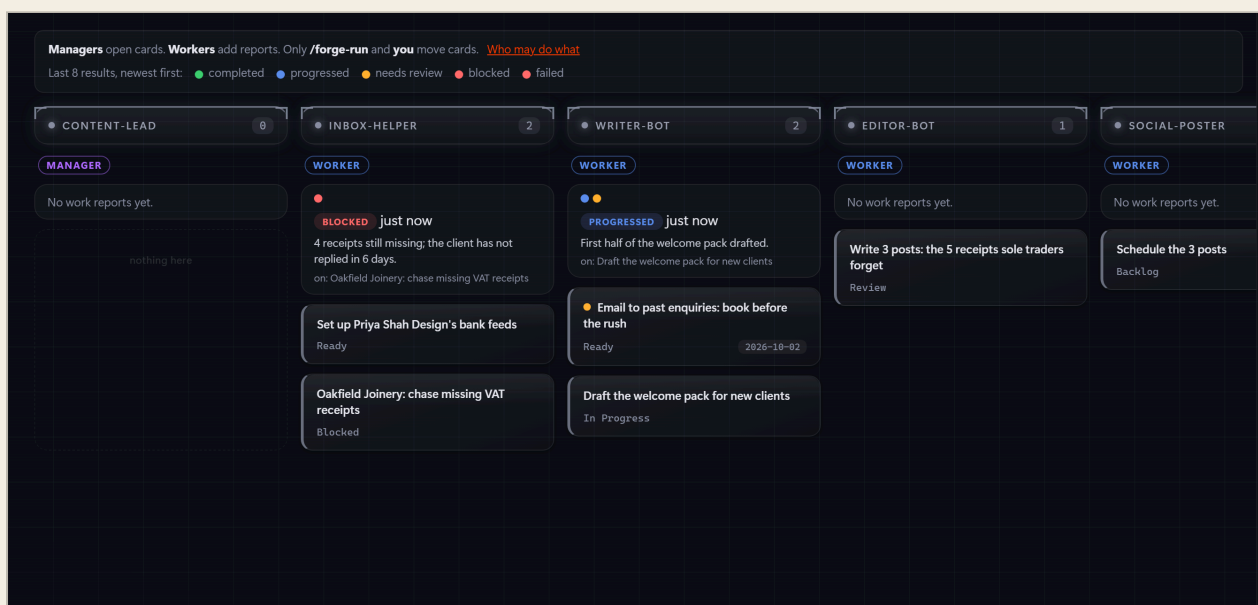
A CARD: THE WORK REPORT WITH THE 3 FILES IT MADE, AND A FULL 5-FIELD HANDOVER.

The **Queue tab** shows the Ready cards in the order they would be handed out (priority, then due date, then age) and says why a card cannot go: it has no owner, it is yours, its owner is not one of your agents, or the in-progress limit is reached.



THE QUEUE: 2 CARDS CAN START NOW, 1 IS YOURS, 1 NEEDS AN OWNER.

The **Agents** tab shows every agent you installed, managers first, each with a role badge, its open cards and its last 8 results as coloured dots. A key to the dots is at the top.



WORK GROUPED BY AGENT, WITH THE ROLE OF EACH AND THE KEY TO THE DOTS.

Your column. Check Awaiting You once a day. Everything there needs you: a decision, a call, a sign-off. Click the "awaiting you" number at the top to jump to it.

Refusals and escalations. The "refused today" number at the top counts the changes the board refused. Click it, or "show refusals only" in Activity, to see who tried what. An agent that needs you can run `forge_agent.py escalate`; that shows in red as NEEDS A PERSON and does not move the card.

▶ ALERTS (1)	
ACTIVITY	show everything
● REFUSED writer-bot: handover refused: too thin done: 'handed over' does not say what was done (name the files); missing decisions: decisions made and why; missing state: where the work stands, including what is NOT done; missing next_first: what the next agent should do first; missing warnings: warnings (write 'none' if there are none) 2026-09-22 14:09:43	
● REFUSED writer-bot: refused: writer-bot is a worker and may not move a card between columns. Workers only append: log a pass, write a handover or a comment. The orchestrator moves cards; a manager opens them. 2026-09-22 14:09:43	
● REFUSED writer-bot: refused: writer-bot is a worker and may not open a card. Workers only append: log a pass, write a handover or a comment. The orchestrator moves cards; a manager opens them. 2026-09-22 14:09:43	

ACTIVITY SHOWING ONLY REFUSALS: WHICH AGENT, WHAT IT TRIED, AND WHY IT WAS REFUSED.

Health check. The board runs a no-AI check when it starts and every 10 minutes: overdue, due soon (2 days), stale (5 days), stuck in Blocked (3 days), no owner, and too many cards in a column. The cards it finds are listed under Alerts on the right (with the time of the last check) and at the top of the summary note. It also sends a card back to Ready if its agent never reported within 45 minutes, and archives Done cards after 14 days. `python forge.py hygiene` runs the same check once and prints the cards.

Letting it run by itself (optional). `python tools/run_if_ready.py` checks first and starts Claude only if a card is ready. It prints 1 line and writes what happened to `data/run_if_ready.log`, not the screen; a pass can take up to 45 minutes. It starts Claude in "dontAsk" mode: nobody is there to answer questions, so it may use exactly the board commands below, plus whatever you have already allowed in your own Claude Code settings, and nothing else. This was tested live on 2026-09-22: the board commands ran and every other command was refused.

What the unattended run is allowed to do

`tools/run_if_ready.py` starts Claude with `--permission-mode dontAsk`: nobody is there to answer a question, so anything not on this list (and not already allowed in your own Claude Code settings) is refused, not asked.

ALLOWED: THE ORCHESTRATOR'S BOARD STEPS

```
forge.py waiting
forge.py next ...
forge.py intake --actor orchestrator
forge.py dispatch <card> --actor orchestrator
forge.py commit ... --actor orchestrator
forge.py move <card> awaiting_you --actor orchestrator
```

ALLOWED: THE AGENTS' TOOL AND STARTING AGENTS

```
forge_agent.py card / pass / handoff
forge_agent.py comment / escalate / mywork
Agent (starts the owner agent)
Read (reads files)
```

REFUSED, TESTED LIVE ON 2026-09-22

```
forge.py move <card> done --actor orchestrator
python -c "print(12345)"
```

and every other command. Tools your agents need for their real work (Write, Edit, web search) come from your own settings, or add them to `schedule.extra_allowed_tools` in `config.json`.

EXACTLY WHAT THE UNATTENDED RUN MAY DO, AND WHAT WAS REFUSED IN THE LIVE TEST.

PIECE 3 • PROJECTFORGE

Fit it to your own AI system

Each idea below comes with a prompt you can paste into Claude Code, opened in the ProjectForge folder. Some ideas use settings you will find in `config.json` after installing; `config.example.json` shows them all filled in with examples.

1. CARDS FROM YOUR CRM'S TODAY PAGE

The download includes `adapters/crm_today.py`. It reads the ranked table on `Today.md` in your CRM vault and makes a card for each person in Awaiting You, linked to that person's note in `People/`. The table needs a rank number, the person's name (matching a file in `People/`), why, and when:

```
| 1 | [[Dan Pike]] | Asked about payroll | Call back today |
```

`[[People/Dan Pike]]` and `[[Dan Pike|Dan]]` work too. Run `--dry-run` first: "(no person note found)" means the name does not match a file in `People/`. Running it again updates the same cards instead of copying them. It never sends anything. On the card, and in the summary note, the person's name opens their note in Obsidian. To hand these cards to a research agent instead of you, set `crm_today.owner` in `config.json`.

From your CRM's Today page to cards

The table the reader needs in Today.md, then a dry run, then the real run. Made-up people.

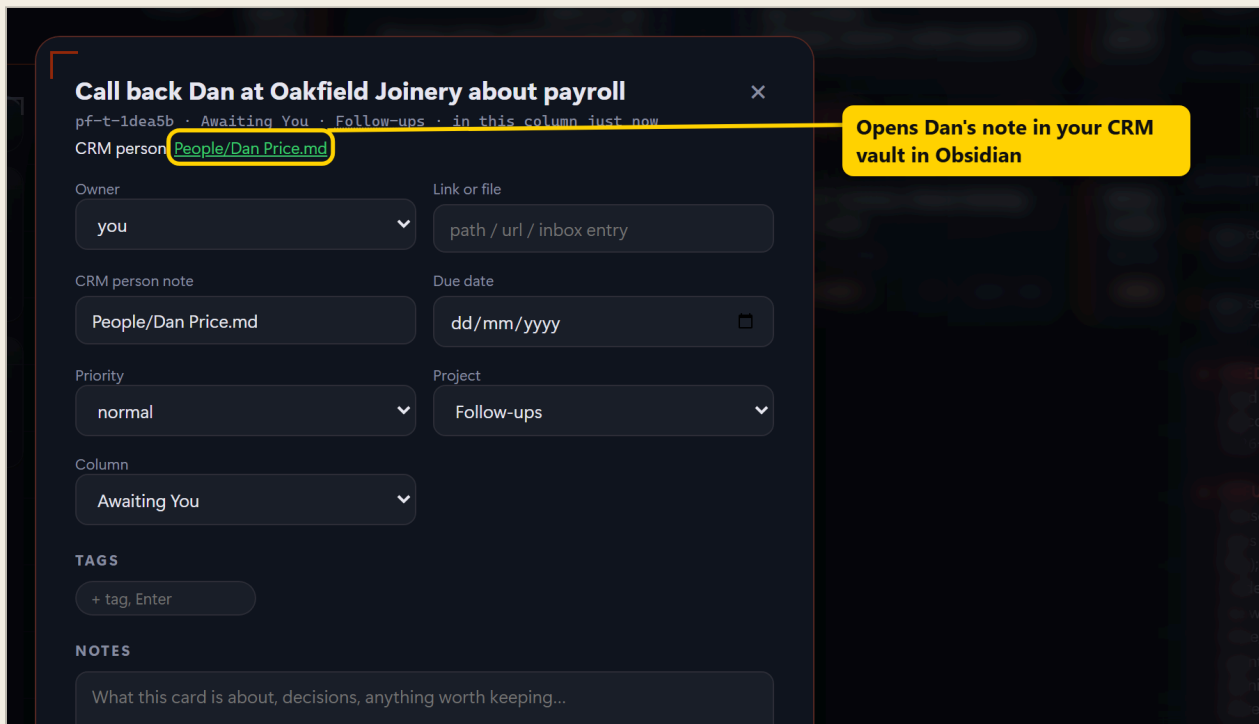
```
CRM vault - Today.md

| # | Who | Why they are here | When | |
|---|---|---|---|---|
| 1 | [[People/Dan Pike]] | Asked about payroll | Call back today |
| 2 | [[Mia Lowe|Mia]] | New connection, runs a design studio | This week |

Terminal

> python adapters/crm_today.py --dry-run
would add: Speak to Dan Pike - Asked about payroll [awaiting_you] People/Dan Pike.md
would add: Speak to Mia Lowe - New connection, runs a design studio [awaiting_you] People/Mia
Lowe.md
2 card(s). Nothing written.
> python adapters/crm_today.py
2 new card(s), 0 already on the board (brought up to date, never copied).
> python adapters/crm_today.py # again: nothing is copied
0 new card(s), 2 already on the board (brought up to date, never copied).
```

THE TODAY.MD TABLE, A DRY RUN, A REAL RUN, AND A SECOND RUN THAT COPIES NOTHING. MADE-UP PEOPLE.



A CRM CARD: THE PERSON LINK OPENS DAN'S NOTE IN THE CRM VAULT.

Run `python adapters/crm_today.py --dry-run` and show me the result. If it looks right, run it for real. Then add one line to my CRM's morning routine so that after Today.md is rebuilt, `crm_today.py` runs straight after it.

2. YOUR SECOND BRAIN AGENTS AS OWNERS AND MANAGERS

Read config.json. Then read every agent file in my agents folder. Suggest which agents should be managers (they plan work and hand it out) and which are workers (they do a single kind of job). Show me the list, and when I agree, re-run python install.py with --managers set to the ones I approve.

3. TEACH EACH AGENT TO LEAVE A PROPER REPORT

Open data/CLAUDE-snippet.md. For each worker agent file in my agents folder, add the WORK REPORT paragraph at the end if it is not already there, and add a line telling it to log that report with forge_agent.py pass. Show me the change for each file before saving it, and back up each file first.

4. CONTENT ENGINE RUNS AS CARDS

Read my content engine folder at <path to your content engine>. Write adapters/content_engine.py, modelled on adapters/crm_today.py, that creates a card per content piece the engine has made in the last 7 days, with the draft's file path in context_ref, owned by my editor agent, starting in Review. It must never publish anything. Add "content-engine" to federate_sources in config.json. Add a test in tests/ and run python -m pytest -q.

5. "AWAITING YOU" IN YOUR DAILY NOTE

Write tools/daily_block.py. It reads the board and writes a section called "Waiting for me" into today's daily note in my second brain (find the daily notes folder by looking at my vault), listing every card in Awaiting You and every card in Review. Replace only that section, never the rest of the note, and write via a temporary file and os.replace. No AI.

6. ROUTE CARDS BY KEYWORD

In config.json, fill intake.routing so that a card with "podcast" in its title goes to my podcast agent, "invoice" goes to my admin agent, and "thumbnail" goes to my design agent. Use the real agent names from config.json "agents". Then add a test to tests/test_store.py proving a card is routed correctly, and run the tests.

7. EVERY CARD MUST NAME THE NUMBER IT MOVES

Ashley's worry about busy-work was agents creating work to stay busy. A cure is to refuse any card that does not say which goal it serves.

Add a "goal" field to cards in engine/db.py (with a migration like the others). Make open_card refuse a card whose goal is empty, with a clear message. Show the goal on the card in the viewer. Add tests for the refusal and run python -m pytest -q.

8. THE OPTIONAL SCHEDULE, SET UP SAFELY

Run `python tools/schedule.py --print` and explain each line to me. Then set `schedule.model` in `config.json` to "sonnet" and `schedule.every_min` to 120. List the tools my agents need for their work that are not in `allowed_tools()` in `tools/run_if_ready.py`, and suggest what to put in `schedule.extra_allowed_tools`. Do not run `python tools/schedule.py --install` until I say yes.

9. LIMITS PER AGENT

The file name decides which agent a limit file applies to: `cards/writer-bot.json` limits writer-bot.

For my 3 busiest worker agents, create a file in `cards/` modelled on `cards/example-agent.json.example`, named after the agent (for example `cards/writer-bot.json`), and set its "slug" to the agent's exact name. Limit each to its own department and to 5 hand-outs a day. Run `python forge.py cards --validate` and show me the result.

PIECE 3 • PROJECTFORGE

Every command and setting

The whole download at a glance, then every command. Anything that changes the board from the terminal needs `--actor <your board name>`.

What is in the download folder

Blue: you run these. Orange: your settings and your board. Green: what Claude Code and your agents use. Grey: the parts behind the scenes.

install.py
asks 8 questions, writes `config.json`, installs the command and the agents' tool

config.json
your settings: names, managers, departments, routing, schedule (written by the installer)

commands/forge-run.md
becomes `/forge-run` in Claude Code: the orchestrator's instructions

adapters/forge_client.py
lets your own programs push cards in (name them in `federate_sources`)

tools/
`run_if_ready.py` (check, then start Claude), `schedule.py` (optional timer), `demo_board.py`

viewer/
the web board: `index.html`, `app.js`, `style.css`

tests/
83 tests: `python -m pytest -q`

forge.py
your terminal commands: `serve`, `list`, `projects`, `add-task`, `move` ... (every change needs `--actor`)

data/forge.db
the board itself: one database file, `data/run_if_ready.log`, `data/CLAUDE-snippet.md` sit beside it

adapters/forge_agent.py
the agents' tool; installed to `.claude/projectforge/`

adapters/crm_today.py
makes cards from your CRM's Today.md

engine/
the rules (`rules.py`, `db.py`), web server, health check, summary note

cards/
optional per-agent limits; `cards/writer-bot.json` limits writer-bot

WHAT EACH FILE AND FOLDER IN THE DOWNLOAD IS FOR.

forge.py : your commands.

COMMAND	WHAT IT DOES
<code>serve [--port 3020]</code>	Opens the web board and runs the health check every 10 minutes.
<code>daemon [--port] [--interval 10]</code>	The same as serve, with its own minutes between health checks.
<code>list</code>	Open cards in the terminal.
<code>projects</code>	Every project and its id.
<code>add-project "Title" --dept content [--summary "..."] --actor you</code>	A new project; prints its id.
<code>add-task <project-id> "Title" [--status ready] [--agent writer-bot] [--notes "..."] [--context <link or file>] [--crm-person "People/Dan Pike.md"] --actor you</code>	A new card. The 8 columns are backlog, ready, in_progress, blocked, review, awaiting_you, done, tracking.
<code>move <card> <column> --actor you</code>	Moves a card.
<code>set <card> [--due 2026-10-02] [--priority low/normal/high/urgent] [--agent <name>] [--crm-person ...] --actor you</code>	Changes a card's details.
<code>archive <card> [--restore] --actor you</code>	Hides a card, or brings it back. Nothing is ever deleted.
<code>comment <card> "text" --actor you</code>	Adds a comment.
<code>pass <card> <agent> "summary" [--result ...] [--output s a.md,b.md] [--next "..."] [--intent ...]</code>	Logs a work report for an agent. Never moves the card.
<code>handoff <card> <from> <to> --done --decisions --state --next-first --warnings [--context]</code>	A 5-field handover from the terminal.
<code>waiting [--json]</code>	How many cards a <code>/forge-run</code> could hand out now, counting owned Backlog cards it will move to Ready first. Writes nothing.
<code>next [--limit 5] [--json]</code>	The Ready cards in hand-out order, with the reason any card cannot go.
<code>show <card> [--json]</code>	A card in full.
<code>metrics [--days 7]</code>	Cards finished, cards sent backwards, how often you stepped in.
<code>hygiene</code>	Runs the health check once and lists the cards it found.
<code>mirror</code>	Rewrites the summary note now.
<code>cards [--validate]</code>	Lists or checks the per-agent limit files.

COMMAND	WHAT IT DOES
<code>intake , dispatch <card> , commit <card> <agent> "summary" --result ... [--key] with --actor orchestrator</code>	The orchestrator's own steps. <code>--key</code> stops the same report being logged twice.

Results are completed, progressed, blocked, failed and needs-review. `--intent` (on pass, handoff and commit) records what kind of message it is: DELEGATE, REQUEST, INFORM, PROPOSE, ACCEPT, REFUSE, FAILURE, QUERY or ESCALATE.

forge_agent.py : your agents' tool. It writes straight into the database, so it works whether or not the web board is open. It finds the board through `forge_agent.json` next to it, or the `FORGE_DIR` setting.

COMMAND	WHO
<code>mywork <agent></code>	Anyone: the agent's open cards and last report.
<code>card <card></code>	Anyone: reads a card, its last 2 handovers and last 4 reports.
<code>pass --card --agent --summary [--result] [--outputs] [--next] [--key]</code>	Any agent.
<code>handoff --card --from --to --done --decisions --state --next-first --warnings [--context]</code>	Any agent. Makes the receiver the owner.
<code>comment --card --agent --text</code>	Any agent.
<code>escalate --card --agent --note</code>	Any agent. Shows in red as NEEDS A PERSON; does not move the card.
<code>open --agent <manager> --dept --project --title [--assignee] [--status backlog/ready] [--context] [--notes] [--crm-person]</code>	Managers only. A new project name makes a new project.

Other scripts. `install.py` also takes every answer as a setting, for scripts: `--yes` , `--second-brain` , `--crm` , `--agents-dir` , `--managers` , `--outward` , `--name` , `--commands` , `--summary-note` , `--port` , and `--uninstall` . `tools/run_if_ready.py [--dry-run]` checks and starts Claude only if needed.

`tools/schedule.py --print / --install [--every 60] / --remove` shows, switches on or switches off the optional schedule (a hidden Windows task, or a launchd job on a Mac). `tools/demo_board.py --out demo [--serve] [--port 3029]` builds the demo. `adapters/forge_client.py` lets your own programs send cards to the board over its web address.

Settings in `config.json` .

SETTING	WHAT IT CONTROLS
<code>workspace</code> , <code>human</code> , <code>orchestrator</code>	The name under the logo, your board name, the orchestrator's name.
<code>managers</code> , <code>agents</code> , <code>outward_owners</code>	Who may open cards; your real agents (the owner picker); whose finished work stops in Review.
<code>departments</code>	The 4 departments, their colours, and an optional <code>lead</code> agent who gets ownerless cards.

SETTING	WHAT IT CONTROLS
<code>hygiene</code>	<code>check_every_min</code> 10, <code>stale_days</code> 5, <code>blocked_days</code> 3, <code>due_soon_days</code> 2, <code>done_archive_days</code> 14, <code>dispatch_ttl_min</code> 45, and the column limits: In Progress 10, Review 8, Awaiting You 10.
<code>intake</code>	<code>auto_pickup</code> , the parking tags, and keyword <code>routing</code> .
<code>crm_today.owner</code>	Give CRM cards to an agent instead of you.
<code>federate_sources</code>	Programs allowed to push cards in (default <code>crm-today</code>). Anything else is refused.
<code>schedule</code>	<code>every_min</code> , <code>model</code> , <code>extra_allowed_tools</code> , and <code>claude_path</code> (saved when you install the schedule).
<code>summary_note</code> , <code>second_brain</code> , <code>crm_vault</code> , <code>port</code> , <code>db_path</code>	Folder paths, the board's port and its database file.

The board itself. Search box and department buttons filter every tab. 5 colour themes (Command Deck, Synthwave, Terminal, Paper and Minimal) sit in the menu at the top. Drag a card between columns, or up and down inside a column; dragging a card that an agent is still working on into Done asks first. A card has tags, a due date, a priority, notes, a checklist and an archive button. The × on an alert dismisses it. Keyboard users can Tab to a card and press Enter to open it.

The web address, for your own programs. The board answers on `http://127.0.0.1:3020/api/...`. Reads: `state`, `events`, `agents`, `alerts`, `metrics`, `cards`, `next`, `waiting`, `task/<card>`. Writes (JSON only, from your own computer, with an `actor`): `open`, `pass`, `handoff`, `federate`, `task/add`, `task/move`, `task/update`, `task/tags`, `task/checklist`, `task/comment`, `task/archive`, `task/reorder`, `project/add`, `project/update`, `alert/dismiss`, and the orchestrator's `intake`, `dispatch`, `commit`.

Also in the folder: `README.md`, `WHAT-I-STOLE.md` (the ideas this borrows and their licences), `LICENSE` (MIT), `guide/` (this guide and its pictures), and `tests/` (83 tests: `python -m pytest -q`).

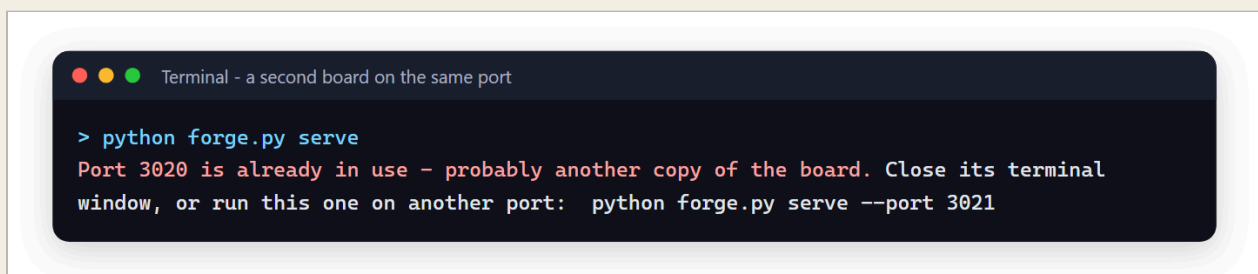
PIECE 3 • PROJECTFORGE

When it goes wrong

These are the real faults from Ashley's build and from testing this download, with the fix for each.

WHAT YOU SEE	WHY	FIX
Your Claude usage climbs while nothing gets done.	A timer is starting Claude on an empty board. This was Ashley's biggest automatic cost in July 2026.	Never put <code>claude -p "/forge-run"</code> on a timer. Use <code>tools/run_if_ready.py</code> , which exits without starting Claude when nothing is ready.
Nothing is ever handed out, but cards sit in Ready.	The in-progress limit (default 10) is full, or the owner is not one of your agents. In the original, 18 mirror cards filled the limit.	Open the Queue tab: it says why each card is waiting. Put cards that only report a fact from another system in Tracking, which never counts.

WHAT YOU SEE	WHY	FIX
A card jumps back to an older column.	Another program re-sent it with its old column.	Already fixed: once the orchestrator claims a card, or while it is in Awaiting You, a re-send can no longer change its column.
"refused: ... is a worker and may not move a card"	The single-writer rule. An agent tried to move a card.	Working as intended. The agent should log a report with <code>forge_agent.py pass ; /forge-run</code> moves the card.
"handover refused - a handover must carry all 5 fields"	A thin handover.	Fill in <code>--done</code> (name the files), <code>--decisions</code> (with "because"), <code>--state</code> , <code>--next-first</code> and <code>--warnings</code> ("none" is fine).
"the following arguments are required: --actor"	Every <code>forge.py</code> change must say who made it.	Add <code>--actor</code> and your board name, for example <code>--actor you</code> .
"no project with the id ..."	A wrong project id.	<code>python forge.py projects</code> lists them.
"unknown department 'marketing'"	Only the departments in <code>config.json</code> are accepted.	Use content, sales, delivery or operations, or add a department to <code>config.json</code> .
"Port 3020 is already in use"	A second copy of the board, or another program, has the port.	Close the other board's terminal window, or run <code>python forge.py serve --port 3021</code> .
Your vault note was overwritten by a demo board.	During research on 2026-09-22, a copied settings file still pointed at a real vault note. It was restored from git.	The demo board never writes a summary note, and the installer shows the note's path before writing it.
The orchestrator reports "bad json".	In the original, an em dash typed into a command's text broke the request.	<code>/forge-run</code> now uses <code>forge.py</code> commands instead of web requests. Still type hyphens, not em dashes.
An agent says the board was not found.	<code>forge_agent.json</code> is missing next to the tool.	Re-run <code>python install.py</code> . Or set the environment variable <code>FORGE_DIR</code> to the download folder.
The unattended run logs "claude not found" (Mac).	A schedule starts with a bare list of program folders.	<code>python tools/schedule.py --install</code> saves Claude's full path and adds its folder to the schedule.



WHAT A SECOND COPY OF THE BOARD SAYS NOW. BEFORE THE FIX, ON WINDOWS, IT STARTED SILENTLY ON THE SAME PORT.

Download

The code: <https://github.com/OUTLIERS-ai/outliers-ws-03-projectforge>

Clone and install with a single command:

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-03-projectforge && cd outliers-ws-03-projectforge && python install.py
```

From nothing to your first card in 3 steps

About 5 minutes. Nothing is installed from the internet except the download itself.

1. DOWNLOAD AND INSTALL

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-03-projectforge
cd outliers-ws-03-projectforge
python install.py
```



2. OPEN THE BOARD

```
python forge.py serve
then visit http://127.0.0.1:3020. Leave the
window open; Ctrl+C stops it.
```



3. FIRST CARD, FIRST RUN

+ Project, then + Add card under Ready. In
Claude Code:
/forge-run dry

FROM NOTHING TO YOUR FIRST CARD IN 3 STEPS.

DOWNLOAD THIS PIECE

<https://github.com/OUTLIERS-ai/outliers-ws-03-projectforge>

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-03-projectforge
cd outliers-ws-03-projectforge
python install.py
```